

DBMS – SELLER AND INVENTORY MANAGEMENT

Project Title: EcoCart – Sustainable Products E-Commerce Order Management System

Skill Task: DBMS

Week: 11

Transaction Management System

Objectives

1. Implement order placement transactions.
2. Apply COMMIT and ROLLBACK operations.
3. Demonstrate ACID properties.
4. Handle payment and stock update transactions.
5. Ensure database consistency.

1. Implement Order Placement Transaction

Insert an Order

```
SQL> INSERT INTO orders
  2  (order_id, customer_id, order_date, total_amount)
  3  VALUES
  4  (105, 101, SYSDATE, 5000);

1 row created.
```

Check the order:

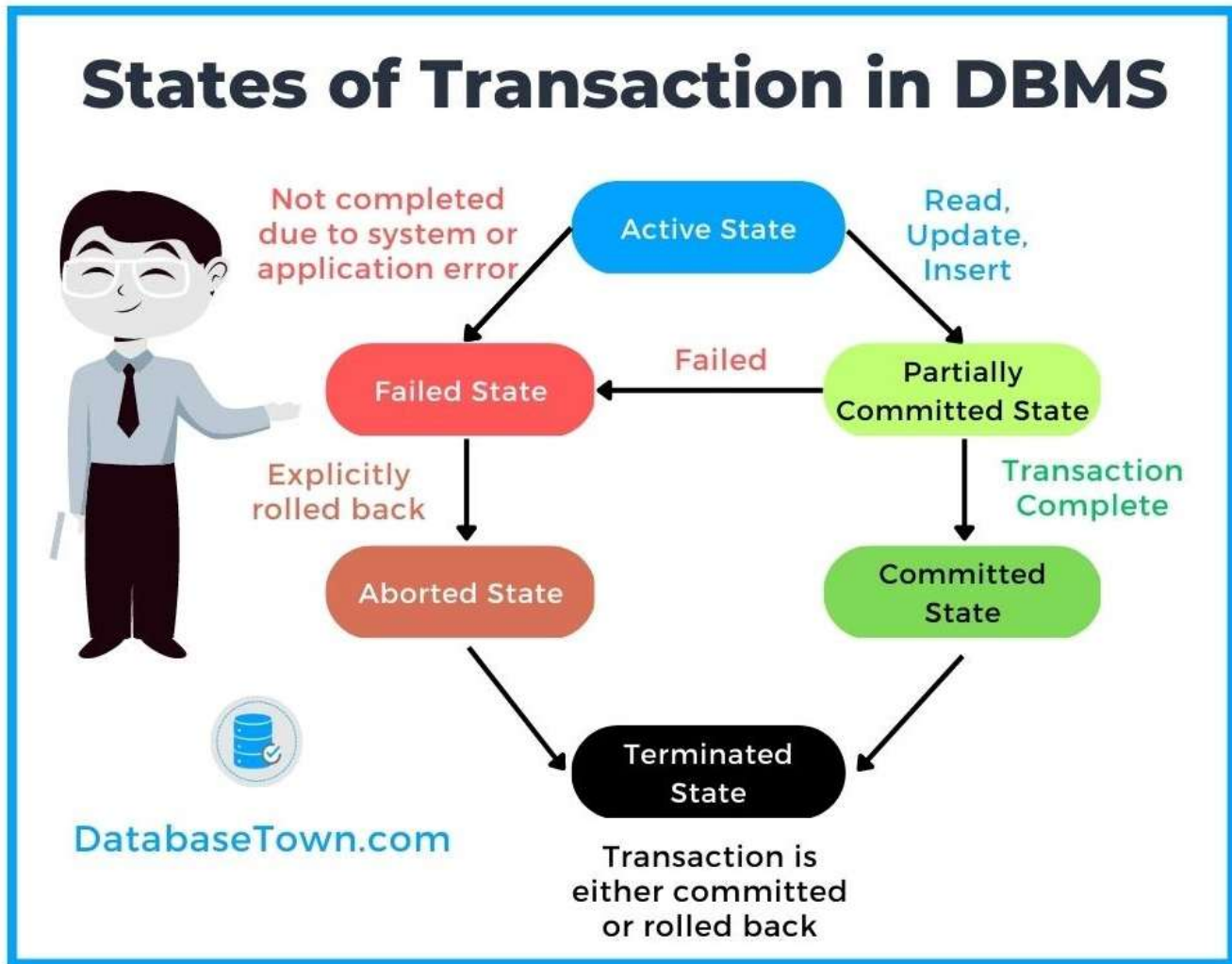
```
SQL> SELECT * FROM orders
  2  WHERE order_id = 105;

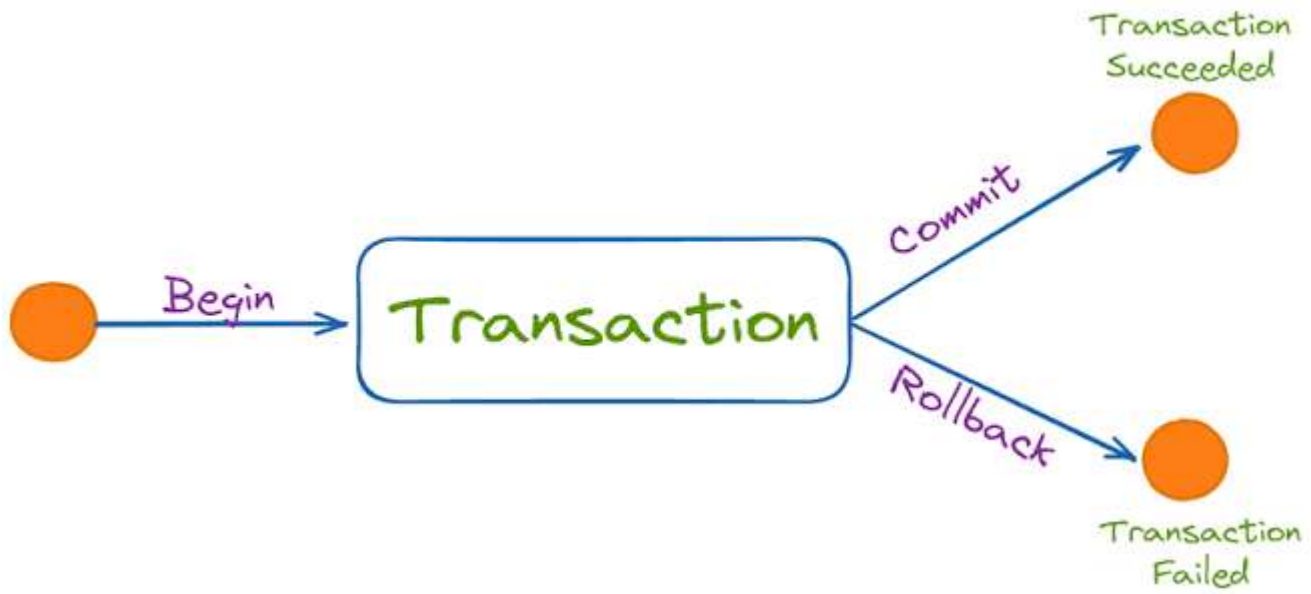
ORDER_ID CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT
-----
      105          101 06-OCT-26          5000
```

At this stage, the transaction is not permanently saved until COMMIT is executed.

What is Transaction?

A transaction is a series of operations that are executed as a single unit of work, ensuring that either all the operations are successfully completed or none of them are. Here are six states of transactions in DBMS





2. Apply COMMIT Operation

COMMIT permanently saves the changes made in the transaction.



```
SQL> COMMIT;
Commit complete.
```

```
SQL> SELECT * FROM orders
2 WHERE order_id = 105;

ORDER_ID  CUSTOMER_ID  ORDER_DAT  TOTAL_AMOUNT
-----
105       101  06-OCT-26      5000
```

3. Apply ROLLBACK Operation

```
SQL> INSERT INTO orders
  2 (order_id, customer_id, order_date, total_amount)
  3 VALUES
  4 (106, 102, SYSDATE, 7000);

1 row created.
```

```
SQL> SELECT * FROM orders
  2 WHERE order_id = 106;
```

ORDER_ID	CUSTOMER_ID	ORDER_DAT	TOTAL_AMOUNT
106	102	06-OCT-26	7000

Now cancel the transaction:

```
SQL> ROLLBACK;

Rollback complete.
```

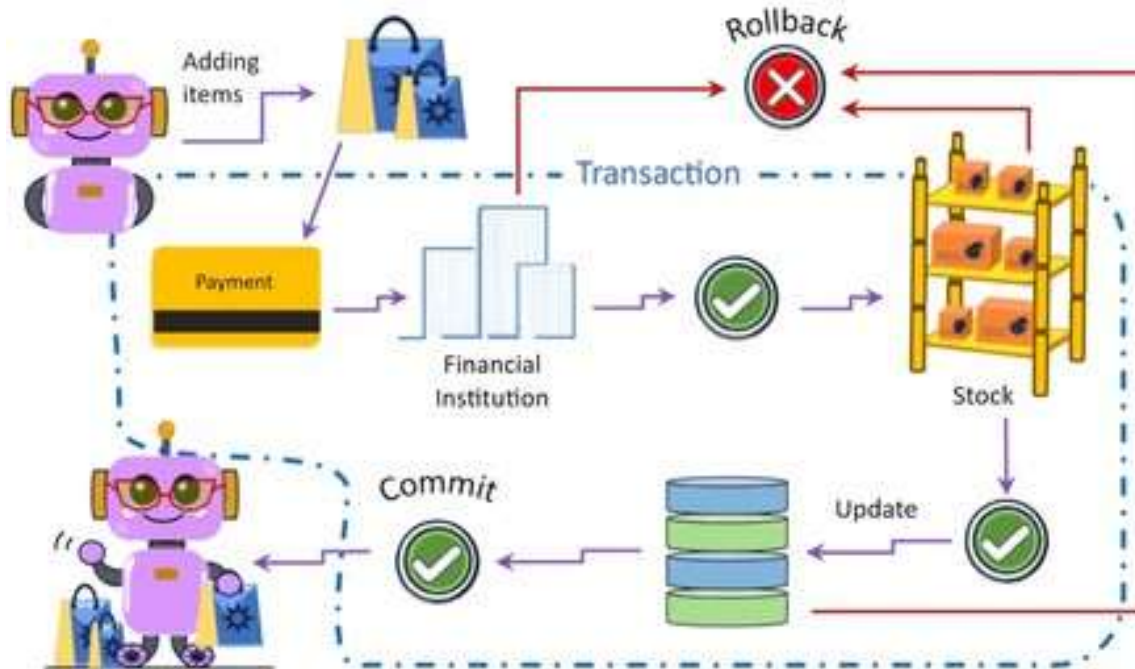
```
SQL> SELECT * FROM orders
  2 WHERE order_id = 106;
```

ORDER_ID	CUSTOMER_ID	ORDER_DAT	TOTAL_AMOUNT
106	102	06-OCT-26	7000

The order will no longer appear because the INSERT was rolled back.

```
SQL> SELECT * FROM orders WHERE order_id = 106;

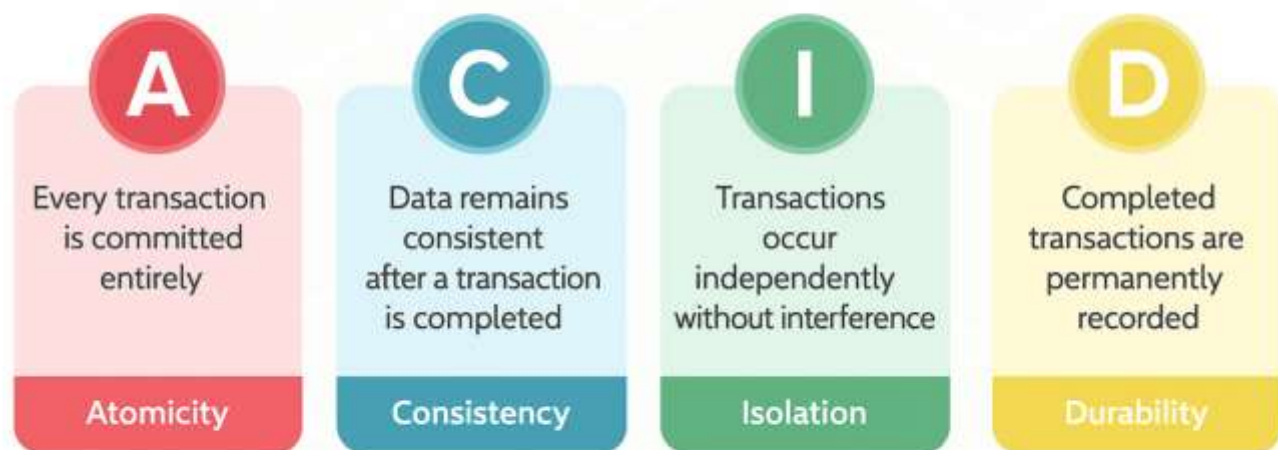
no rows selected
```



shutterstock.com · 2567926801

4. Demonstrate ACID Properties

ACID Properties in DBMS



A – Atomicity

A transaction is treated as a single unit.

```
SQL> INSERT INTO orders
  2 (order_id, customer_id, order_date, total_amount)
  3 VALUES
  4 (107, 101, SYSDATE, 8000);

1 row created.
```

```
SQL> ROLLBACK;

Rollback complete.
```

The complete operation is cancelled.

C – Consistency

The database should remain valid before and after a transaction.

For example, ORDER_ID is a primary key, so duplicate order IDs are not allowed.

```
SQL> INSERT INTO orders
  2 (order_id, customer_id, order_date, total_amount)
  3 VALUES
  4 (105, 101, SYSDATE, 5000);
INSERT INTO orders
*
ERROR at line 1:
ORA-00001: unique constraint (SYSTEM.SYS_C007079) violated
```

If 105 already exists, Oracle will reject the operation.

I – Isolation

One transaction's uncommitted changes should not interfere with another transaction.

```
SQL> INSERT INTO orders
  2 (order_id, customer_id, order_date, total_amount)
  3 VALUES
  4 (108, 103, SYSDATE, 9000);

1 row created.
```

Until:

```
SQL> COMMIT;

Commit complete.
```

the change remains part of the current transaction.

D – Durability

After:

```
SQL> COMMIT;

Commit complete.
```

the changes are permanently stored in the database.

```
SQL> SELECT * FROM orders WHERE order_id = 108;

ORDER_ID CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT
-----
108      103 06-OCT-26      9000
```

5. Payment and Stock Update Transaction

we need PAYMENT and INVENTORY tables.

Example Payment Table

```
SQL> DESC Payment;
Name                               Null?    Type
-----
PAYMENT_ID                         NOT NULL NUMBER(5)
ORDER_ID                           NOT NULL NUMBER(5)
PAYMENT_MODE                       NOT NULL VARCHAR2(20)
PAYMENT_DATE                       NOT NULL DATE
PAYMENT_AMOUNT                     NOT NULL NUMBER(10,2)
PAYMENT_STATUS                     NOT NULL VARCHAR2(20)
```

Example Inventory Table

```
SQL> DESC Inventory;
Name                               Null?    Type
-----
INVENTORY_ID                       NOT NULL NUMBER(38)
PRODUCT_ID                         NOT NULL NUMBER(38)
SELLER_ID                          NOT NULL NUMBER(38)
STOCK_QUANTITY                     NOT NULL NUMBER(38)
STOCK_STATUS                       NOT NULL VARCHAR2(20)
LAST_UPDATED                       NOT NULL DATE
```

Transaction: Order + Payment

```
SQL> INSERT INTO orders
  2  (order_id, customer_id, order_date, total_amount)
  3  VALUES
  4  (109, 101, SYSDATE, 5000);

1 row created.
```

```
SQL> INSERT INTO payment
  2      (payment_id,
  3      order_id,
  4      payment_mode,
  5      payment_date,
  6      payment_amount,
  7      payment_status)
  8  VALUES
  9      (1,
 10      109,
 11      'UPI',
 12      SYSDATE,
 13      5000,
 14      'SUCCESS');
```

1 row created.

```
SQL> COMMIT;

Commit complete.
```

Both operations are saved.

```
SQL> SELECT *
  2  FROM orders
  3  WHERE order_id = 109;

  ORDER_ID  CUSTOMER_ID  ORDER_DAT  TOTAL_AMOUNT
-----
      109           101  06-OCT-26           5000

SQL> SELECT *
  2  FROM orders
  3  WHERE order_id = 109;

  ORDER_ID  CUSTOMER_ID  ORDER_DAT  TOTAL_AMOUNT
-----
      109           101  06-OCT-26           5000
```

6. Payment Failure Using ROLLBACK

Suppose payment processing fails.

```
SQL> INSERT INTO orders
  2     (order_id, customer_id, order_date, total_amount)
  3  VALUES
  4     (110, 102, SYSDATE, 6000);
```

1 row created.

```
SQL> INSERT INTO payment
  2     (payment_id,
  3      order_id,
  4      payment_mode,
  5      payment_date,
  6      payment_amount,
  7      payment_status)
  8  VALUES
  9     (2,
 10      110,
 11      'CARD',
 12      SYSDATE,
 13      6000,
 14      'FAILED');
```

1 row created.

```
SQL> ROLLBACK;
```

Rollback complete.

Now both changes are cancelled.

```
SQL> SELECT * FROM orders
  2  WHERE order_id = 110;
```

no rows selected

```
SQL> SELECT * FROM payment
  2  WHERE order_id = 110;
```

no rows selected

No record should be returned.

7. Stock Update Transaction

```
SQL> SELECT product_id,  
2          stock_quantity,  
3          stock_status  
4 FROM inventory  
5 WHERE product_id = 101;
```

PRODUCT_ID	STOCK_QUANTITY	STOCK_STATUS
101	45	AVAILABLE

```
SQL> UPDATE inventory  
2 SET stock_quantity = stock_quantity - 2,  
3     stock_status =  
4     CASE  
5         WHEN stock_quantity - 2 = 0  
6         THEN 'OUT OF STOCK'  
7         ELSE 'AVAILABLE'  
8     END,  
9     last_updated = SYSDATE  
10 WHERE product_id = 101  
11     AND stock_quantity >= 2;
```

1 row updated.

```
SQL> SELECT product_id,  
2          stock_quantity,  
3          stock_status  
4 FROM inventory  
5 WHERE product_id = 101;
```

PRODUCT_ID	STOCK_QUANTITY	STOCK_STATUS
101	43	AVAILABLE

Test ROLLBACK

To demonstrate transaction management, make another change:

```
SQL> UPDATE inventory
  2 SET stock_quantity = stock_quantity - 5,
  3     last_updated = SYSDATE
  4 WHERE product_id = 101;
```

1 row updated.

```
SQL> SELECT product_id, stock_quantity
  2 FROM inventory
  3 WHERE product_id = 101;
```

PRODUCT_ID	STOCK_QUANTITY
101	38

```
SQL> ROLLBACK;
```

Rollback complete.

```
SQL> SELECT product_id, stock_quantity
  2 FROM inventory
  3 WHERE product_id = 101;
```

PRODUCT_ID	STOCK_QUANTITY
101	43

8. Ensure Database Consistency

Database consistency is maintained by ensuring that related operations are completed together. For example, when an order is successfully placed, the corresponding payment is recorded and the inventory stock is reduced. If any operation fails, ROLLBACK is used to cancel the uncommitted changes. Thus, incomplete transactions do not leave the database in an invalid state.